

ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅԱՆ ԿՐԹՈՒԹՅԱՆ ԵՎ
ԳԻՏՈՒԹՅԱՆ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ
ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆԻ ԳՅՈՒՄՐՈՒ
ՄԱՍՆԱՃՅՈՒՂ

Լաբորատոր աշխատանք

Առարկա՝ Օբյեկտ կողմնորոշված ծրագրավորում

Խումբ՝ ԳԻԿ 519Բ

Կուրս՝ 2-րդ **Ուսանող** Յուրա Իկիլիկյան

Դասախոս՝ Սոնյա Սահակյան

ԳՅՈՒՄՐԻ - 2026

Լաբորատոր Աշխատանք 5

13 Գիտաժողովի զեկույցների ժամանակացույց

Աշխատանքի նպատակ. Առաջադրանքները նախատեսված են կառուցվածքային տիպի զանգվածների օգտագործումը ֆունկցիաներում թեմայի ամրապնդման համար:

Խնդրի պահանջ. Յուրաքանչյուր խնդրի համար սահմանել համապատասխան struct, ստեղծել կառուցվածքների զանգված, իսկ հիմնական հաշվարկները, որոնումները, փոփոխությունները և դասավորումները կատարել առանձին ֆունկցիաներով:

Առաջադրանք. Ստեղծել Report կառուցվածք՝ speaker, topic, hall, startHour, duration դաշտերով: Մուտքագրել 10 զեկույց:

Ֆունկցիաների միջոցով՝

- հաշվել յուրաքանչյուր զեկույցի ավարտի ժամը,
- արտածել տրված hall-ի զեկույցները,
- գտնել ամենաերկար զեկույցը,
- ստուգել՝ արդյոք նույն hall-ում երկու զեկույցների ժամանակները հատվում են,
- դասակարգել զեկույցներն ըստ startHour-ի:

```
#include <iostream>
#include <string>
```

```
using namespace std;
```

```
struct Report {
    string speaker;
    string topic;
    string hall;
    double startHour;
    double duration;
};
```

```
void calculateEndTime(const Report reports[], int size) {
    cout << "\n--- Չեկույցների ավարտի ժամերը ---\n";
```

```

    for (int i = 0; i < size; i++) {
        double endHour = reports[i].startHour + reports[i].duration;
        cout << i + 1 << ". " << reports[i].topic << " - Ավարտը: " <<
endHour << " ժ\n";
    }
}

```

```

void printByHall(const Report reports[], int size, const string&
targetHall) {
    cout << "\n--- '" << targetHall << "' դահլիճի գեկույցները ---\n";
    bool found = false;
    for (int i = 0; i < size; i++) {
        if (reports[i].hall == targetHall) {
            cout << "- " << reports[i].topic << " (Չեկույցող: " <<
reports[i].speaker << ", Սկիզբը: " << reports[i].startHour << " ժ)\n";
            found = true;
        }
    }
    if (!found) cout << "Այս դահլիճում գեկույցներ չեն գտնվել:\n";
}

```

```

void findLongestReport(const Report reports[], int size) {
    if (size == 0) return;
    int maxIndex = 0;
    for (int i = 1; i < size; i++) {
        if (reports[i].duration > reports[maxIndex].duration) {
            maxIndex = i;
        }
    }
    cout << "\n--- Ամենաերկար գեկույցը ---\n";
    cout << "Թեմա: " << reports[maxIndex].topic
        << " | Չեկույցող: " << reports[maxIndex].speaker
        << " | Տևողությունը: " << reports[maxIndex].duration << " ժ\n";
}

```

```

void checkOverlaps(const Report reports[], int size) {
    cout << "\n--- Ստուգում ըստ դահլիճների (Ժամանակների
հասումներ) ---\n";
    bool overlapFound = false;
    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            if (reports[i].hall == reports[j].hall) {
                double end1 = reports[i].startHour + reports[i].duration;
                double end2 = reports[j].startHour + reports[j].duration;

                if (reports[i].startHour < end2 && reports[j].startHour < end1)
{
                    cout << "Հասում հայտնաբերվել է '" << reports[i].hall <<
"' դահլիճում հետևյալ գեկույցների միջև:\n"

```

```

        << " 1. " << reports[i].topic << " (" <<
reports[i].startHour << " - " << end1 << ")\n"
        << " 2. " << reports[j].topic << " (" <<
reports[j].startHour << " - " << end2 << ")\n\n";
        overlapFound = true;
    }
}
}
}
if (!overlapFound) {
    cout << "Նույն դահլիճում ժամանակային հատվող զեկույցներ
չկան:\n";
}
}

void sortByStartHour(Report reports[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (reports[j].startHour > reports[j + 1].startHour) {
                Report temp = reports[j];
                reports[j] = reports[j + 1];
                reports[j + 1] = temp;
            }
        }
    }
    cout << "\n--- Չեկույցները՝ դասակարգված ըստ սկզբի ժամի ---\n";
    for (int i = 0; i < size; i++) {
        cout << i + 1 << ". Սկիզբը: " << reports[i].startHour << " ժ |
Թեմա: " << reports[i].topic << " (Դահլիճ: " << reports[i].hall << ")\n";
    }
}

int main() {
    const int SIZE = 10;
    Report reports[SIZE];

    cout << "Մուտքագրեք " << SIZE << " զեկույցների տվյալները:\n";
    for (int i = 0; i < SIZE; i++) {
        cout << "\nՉեկույց " << i + 1 << ":\n";
        cout << "Չեկույցող (speaker): ";
        getline(cin >> ws, reports[i].speaker);
        cout << "Թեմա (topic): ";
        getline(cin, reports[i].topic);
        cout << "Դահլիճ (hall): ";
        getline(cin, reports[i].hall);
        cout << "Սկզբի ժամ (startHour, օրինակ՝ 10.5): ";
        cin >> reports[i].startHour;
        cout << "Տևողությունը ժամերով (duration): ";
        cin >> reports[i].duration;
    }
}

```

```
}  
  
calculateEndTime(reports, SIZE);  
  
string targetHall;  
cout << "\nՄուտքագրեք դահլիճի անունը որոնման համար: ";  
getline(cin >> ws, targetHall);  
printByHall(reports, SIZE, targetHall);  
  
findLongestReport(reports, SIZE);  
  
checkOverlaps(reports, SIZE);  
  
sortByStartHour(reports, SIZE);  
  
return 0;  
}
```